

Введение

Университет хочет отслеживать популярность различных программ с течением времени, чтобы гарантировать, что программам выделено достаточно места и ресурсов. Вы работаете в приемной комиссии, и вас просят составить набор наглядных материалов, демонстрирующих эти тенденции заинтересованным абитуриентам. Как мы можем рассчитать эти распределения? Сможем ли мы увидеть тенденции и спрогнозировать популярность определенных программ в будущем? Как бы мы показали эту информацию?

В этом уроке мы узнаем, как использовать NumPy для анализа различных распределений, генерировать случайные числа для создания наборов данных.

Этот урок будет охватывать:

- Как идентифицировать разные распределения по их форме
- Нормальные распределения
- Как стандартные отклонения соотносятся с нормальным распределением
- Биномиальные распределения

Задание

Ознакомьтесь с примером ниже

Статистические распределения с NumPy

Представьте, что вы работаете в приемной комиссии в университете. Часть вашей работы - собирать, анализировать и визуализировать данные, которые имеют отношение к заинтересованным кандидатам.

Недавно вы заинтересовались, как гистограммы могут отображать различные распределения популяций и даже встречаемости. Вы думаете, что гистограммы были бы полезны для визуализации различных тенденций, таких как изменение количества отделений и участие во внеклассных занятиях. Вы также хотите узнать больше о том, как вы можете использовать случайно сгенерированные распределения для статистических расчетов и прогнозирования вероятности будущих событий, таких как успех вашей окончательной команды фрисби.

В этом уроке мы будем использовать NumPy для вычисления распределений и Matplotlib для построения графиков наших вычислений.

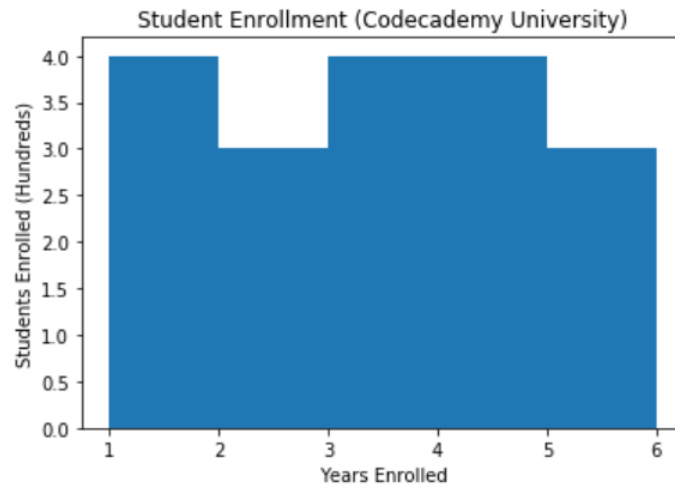
```
In [2]: import numpy as np
        from matplotlib import pyplot as plt
```

Один набор данных, который вы хотите проанализировать, — это зачисление на разные программы обучения. Посмотрев на гистограммы количества лет, в течение которых студенты обучаются по программе, вы можете определить, какие программы становятся более популярными, какие теряют популярность, а какие имеют устойчивый, непрерывный набор.

Во-первых, давайте посмотрим, сколько сотен студентов решают поступить в университет и сколько лет они учатся.

```
In [3]: total_enrollment = [1, 1, 1, 1, 2, 2, 2, 3, 3, 3, 3, 4, 4, 4, 4, 5, 5, 5]

plt.hist(total_enrollment, bins=5, range=(1, 6))
plt.title('Student Enrollment (Codecademy University)')
plt.xlabel('Years Enrolled')
plt.ylabel('Students Enrolled (Hundreds)')
plt.show()
```

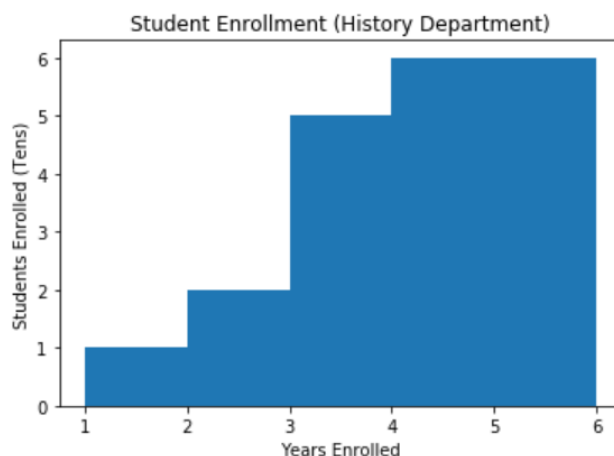


Гистограмма выше показывает общее количество учащихся университета, которое довольно стабильно. Это равномерное распределение, и это то, что хочет видеть университет. Общее количество учащихся остается на хорошем уровне.

Теперь давайте посмотрим на набор специально для студентов, желающих получить степень по истории:

```
In [4]: history_enrollment = [1, 2, 2, 3, 3, 3, 3, 3, 4, 4, 4, 4, 4, 4, 5, 5, 5, 5, 5, 5]

plt.hist(history_enrollment, bins=5, range=(1, 6))
plt.title('Student Enrollment (History Department)')
plt.xlabel('Years Enrolled')
plt.ylabel('Students Enrolled (Tens)')
plt.show()
```

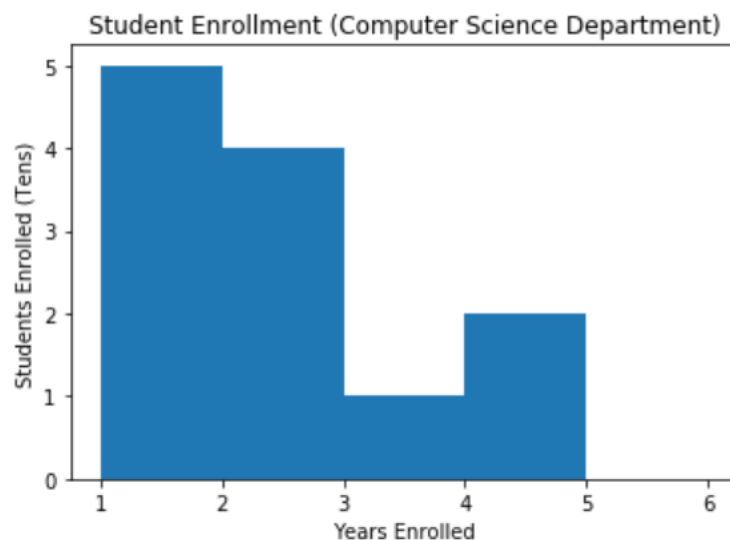


О чем говорит нам эта гистограмма? Что ж, это несколько искаженный левый набор данных, мы видим, что намного больше студентов, которые обучались на 3 или 4 года за 1 и 2 года. Это говорит о том, что программа «История» становится все менее популярной. Куда же тогда собираются все студенты?

Школа недавно вложила много денег в новое здание факультета компьютерных наук. Давайте посмотрим на набор и посмотрим, окупаются ли вложения.

```
In [5]: cs_enrollment = [1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 3, 4, 4]

plt.hist(cs_enrollment, bins=5, range=(1, 6))
plt.title('Student Enrollment (Computer Science Department)')
plt.xlabel('Years Enrolled')
plt.ylabel('Students Enrolled (Tens)')
plt.show()
```



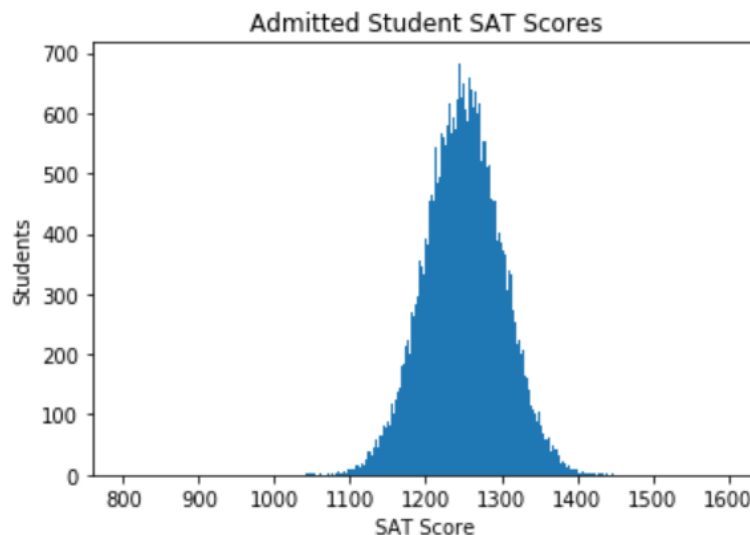
Похоже, что за последние годы количество студентов на факультете компьютерных наук резко возросло. Это могло быть связано с тем, что университет вложил средства в отдел, или это могло быть признаком того, что востребованные профессиональные навыки в реальном мире меняются. Какой бы ни была причина, гистограммы позволяют четко видеть тенденции.

Заинтересованные абитуриенты хотели бы знать, какие результаты SAT имеют принятые студенты. Ранее вы подсчитали, что средний балл составляет 1250 со стандартным отклонением 50.

Вместо того, чтобы собирать оценки каждого учащегося, вы берете то, что знаете о данных, и используете генератор случайных чисел для создания модели.

```
In [6]: sat_scores = np.random.normal(1250, 50, size=100000)

plt.hist(sat_scores, bins=1000, range=(800,1600))
plt.title('Admitted Student SAT Scores')
plt.xlabel('SAT Score')
plt.ylabel('Students')
plt.show()
```



95% студентов набирают баллы в пределах двух стандартных отклонений от среднего. Заинтересованный студент набирает 1130 и хочет знать, находятся ли они в этом диапазоне.

```
In [7]: mean = 1250
one_std = 50

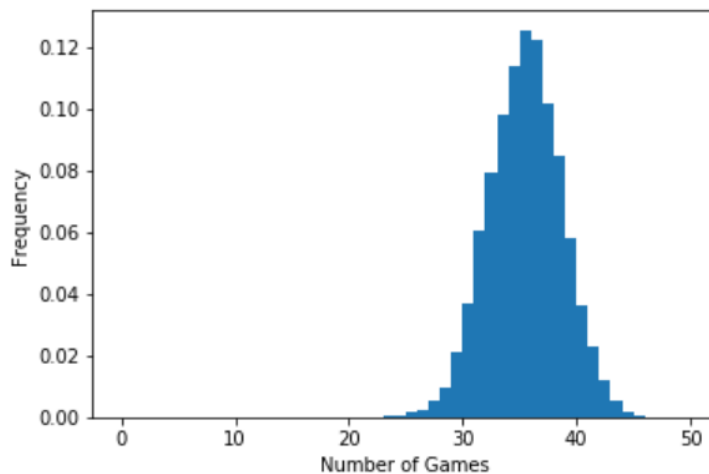
two_below = (mean - 2*one_std)
print two_below

1150
```

Похоже, они прямо под ним! Лучше еще раз пройти этот тест.

Одна из главных составляющих вашей школы - это ваша превосходная команда фрисби. Команда выигрывает около 70% из 50 игр за сезон, или 35 игр. Заинтересованный кандидат хочет знать, каковы шансы, что он сможет улучшить свой рекорд до 40 игр. Вы используете то, что знаете о биномиальных распределениях, чтобы вычислить вероятность такого появления:

```
In [8]: ultimate = np.random.binomial(50, 0.70, size=10000)
plt.hist(ultimate, range=(0, 50), bins=50, normed=True)
plt.xlabel('Number of Games')
plt.ylabel('Frequency')
plt.show()
```



Поскольку это немного сложно увидеть на графике, давайте точно посчитаем, какой у них шанс выиграть 40 игр:

```
In [10]: ultimate = np.random.binomial(50, 0.70, size=10000)
          np.mean(ultimate == 40)
```

```
Out[10]: 0.041000000000000002
```

Гистограммы, часть I

Когда мы впервые смотрим на набор данных, мы хотим быстро понять некоторые вещи о нем:

- Встречаются ли одни ценности чаще, чем другие?
- Каков диапазон набора данных (т. Е. Минимальное и максимальное значения)?
- Много выбросов?

Мы можем визуализировать эту информацию с помощью диаграммы, называемой гистограммой.

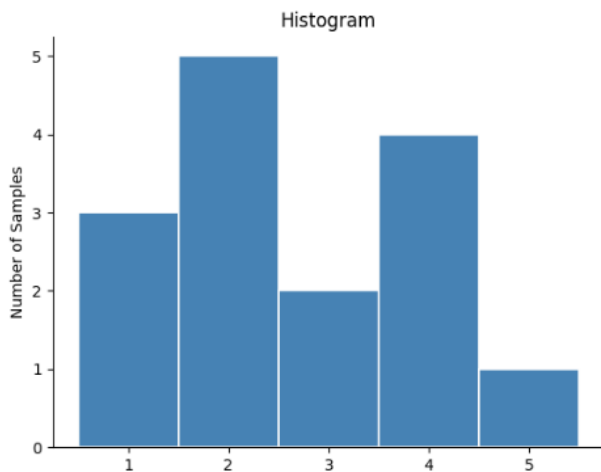
Например, предположим, что у нас есть следующий набор данных:

```
d = [1, 1, 1, 2, 2, 2, 2, 2, 3, 3, 4, 4, 4, 4, 5]
```

Простая гистограмма может показать нам, сколько единиц, двух, трех и т. Д. В этом наборе данных.

Value	Number of Samples
1	3
2	5
3	2
4	4
5	1

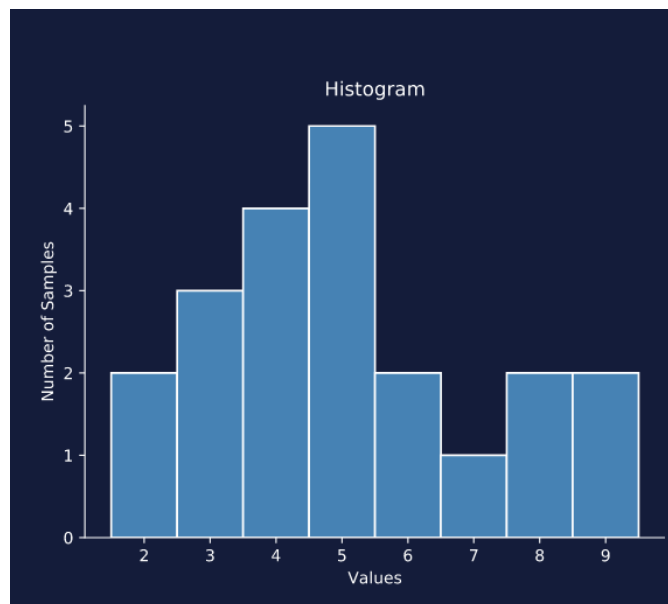
На графике наша гистограмма будет выглядеть так:



Задание

1. Посмотрите на гистограмму справа. Сколько значений в наборе данных равно 5 или 6?

Сохраните сумму в значение переменной `sum`.



Гистограммы, часть II

Предположим, у нас есть более крупный набор данных со значениями от 0 до 50. Возможно, нам не нужно точно знать, сколько у нас нулей, единиц, двоек и т. Д.

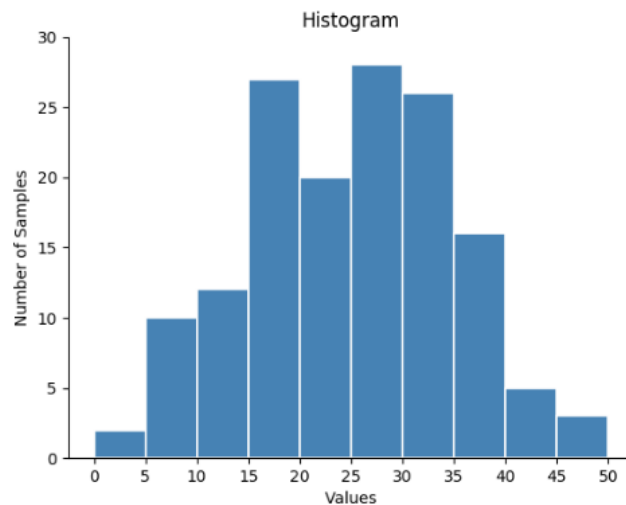
Вместо этого мы могли бы захотеть узнать, сколько значений попадает в интервалы между 0 и 5, 6 и 10, 11 и 15 и т. Д.

Эти группы называются корзинами. Все интервалы на гистограмме всегда одного размера. Ширина каждой ячейки — это расстояние между минимальным и максимальным значениями каждой ячейки. В нашем примере ширина каждой корзины будет равна 5.

Для такого набора данных наша таблица гистограммы будет выглядеть так:

Bin	Number of Values
(0, 5)	2
(6, 10)	10
(11, 15)	11
...	...
(46, 50)	3

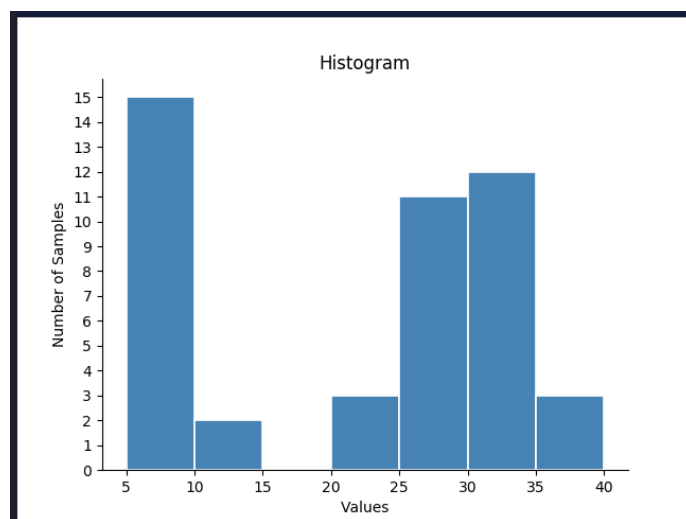
И если бы мы изобразили эту гистограмму, она бы выглядела так:



Задание

1. Представлена новая гистограмма, представляющая новый набор данных.

Какова ширина каждой ячейки на этой гистограмме? Сохраните свой ответ в переменной ширины.



2. Сколько выборок в этом наборе данных находится между 10 и 15? Сохраните свой ответ в переменной ans.

Гистограммы, часть III

Мы можем построить гистограммы с помощью модуля Python, известного как Matplotlib. Более подробно функционал этой библиотеки мы будем рассматривать в отдельной теме.

А пока ознакомьтесь со следующим синтаксисом для построения гистограммы:

```
# This imports the plotting package. We only need to do this
once.
from matplotlib import pyplot as plt

# This plots a histogram
plt.hist(data)

# This displays the histogram
plt.show()
```

Когда мы вводим `plt.hist` без аргументов ключевого слова, `matplotlib` автоматически создает гистограмму с 10 ячейками одинаковой ширины, охватывающими весь диапазон наших данных.

Если вам нужно другое количество ящиков, используйте ячейки ключевых слов. Например, следующий код даст нам 5 ячеек вместо 10:

```
plt.hist(data, bins=5)
```

Если вам нужен другой диапазон, вы можете передать минимальное и максимальное значения, которые вы хотите построить на гистограмме, используя диапазон ключевых слов. Передаем кортеж из двух чисел. Первое число — это минимальное значение, которое мы хотим построить, а второе значение - это число, до которого мы хотим построить график, но не включая его.

Например, если наш набор данных содержит значения от 0 до 100, но мы хотим только гистограмму чисел от 20 до 50, мы могли бы использовать эту команду:

```
# We pass 51 so that our range includes 50
plt.hist(data, range=(20, 51))
```

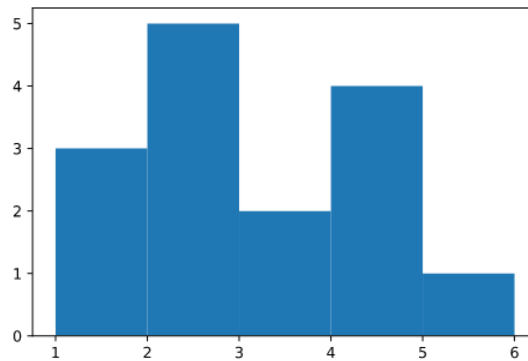
Вот полный пример:

```
from matplotlib import pyplot as plt

d = np.array([1, 1, 1, 2, 2, 2, 2, 2, 3, 3, 4, 4, 4, 4, 5])

plt.hist(d, bins=5, range=(1, 6))

plt.show()
```

Задание

1. Мы включили набор данных, который содержит информацию о времени в пути (в минутах, округленных до ближайшего целого числа) в системе метро Нью-Йорка.

```
commutes = np.array([20, 30, 50, 40, 24, 26, 27, 27, 30, 50, 20, 34, 15,
18, 17, 35, 37, 38, 43, 45, 47])
```

Начните с импорта matplotlib.

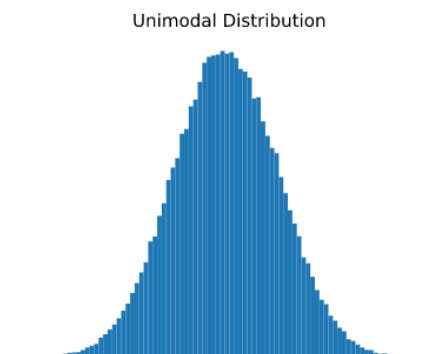
2. Отобразите гистограмму с помощью plt.show ().
3. Теперь предположим, что мы хотели увеличить масштаб и посмотреть, сколько людей ездят на работу от 20 до 50 минут.
Измените команду гистограммы, чтобы получить гистограмму с диапазоном от 20 до 50.
4. Измените гистограмму еще раз, чтобы на ней было 6 интервалов.

Различные типы распределений, часть I

Гистограммы и их наборы данных можно классифицировать по форме отображаемых значений. В следующих двух упражнениях мы рассмотрим два разных способа описания гистограмм.

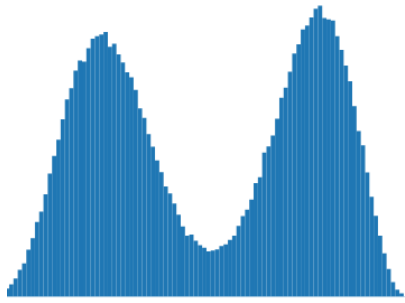
Один из способов классификации набора данных - подсчет количества отдельных пиков, присутствующих на графике. Пики представляют собой концентрации данных. Давайте посмотрим на следующие примеры:

В одномодальном наборе данных есть только один отчетливый пик.

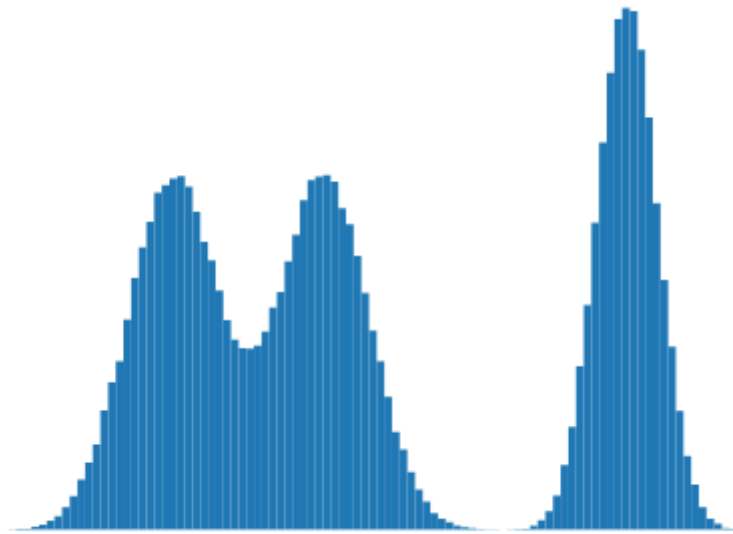


Бимодальный набор данных имеет два различных пика. Это часто случается, когда данные содержат две разные совокупности.

Bimodal Distribution

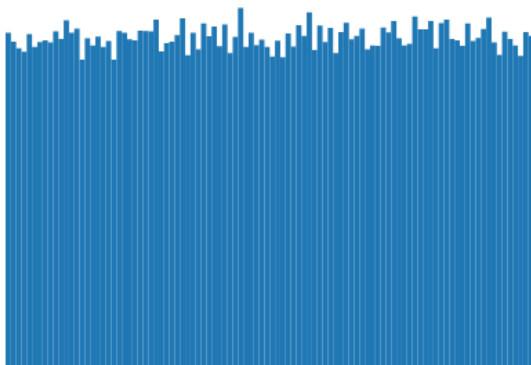


Мультимодальный набор данных имеет более двух пиков.



Единый набор данных не имеет четких пиков.

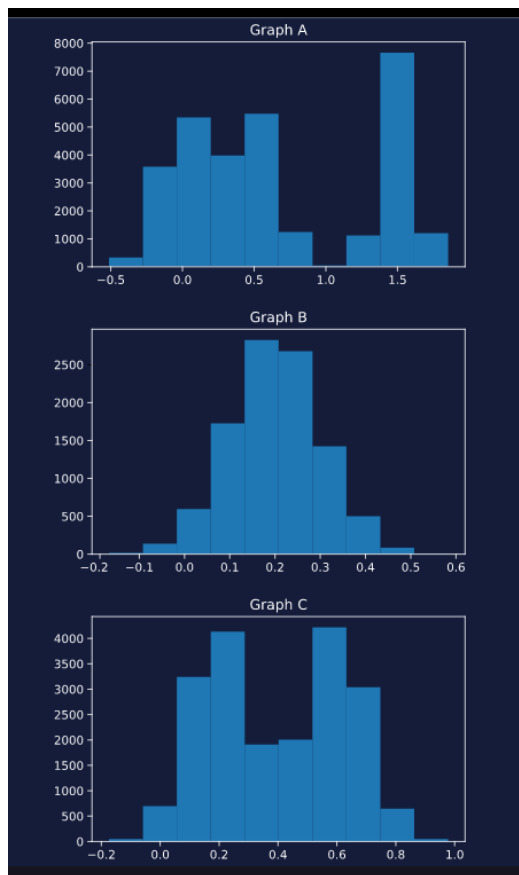
Uniform Distribution



Задание

1. Определите форму распределения Графика А и сохраните свой ответ (в виде строки) в переменной `graph_a`.
2. Определите форму распределения Графика В и сохраните свой ответ (в виде строки) в переменной `graph_b`.

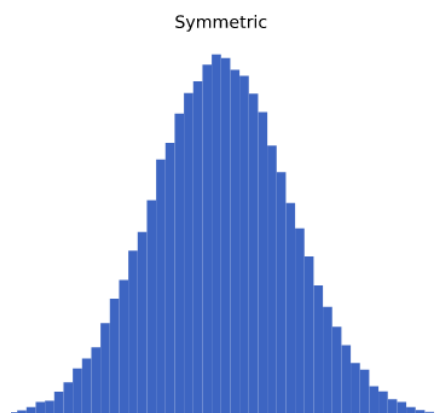
3. Определите форму распределения Графика С и сохраните свой ответ (в виде строки) в переменной `graph_c`.



Различные типы распространения, часть II

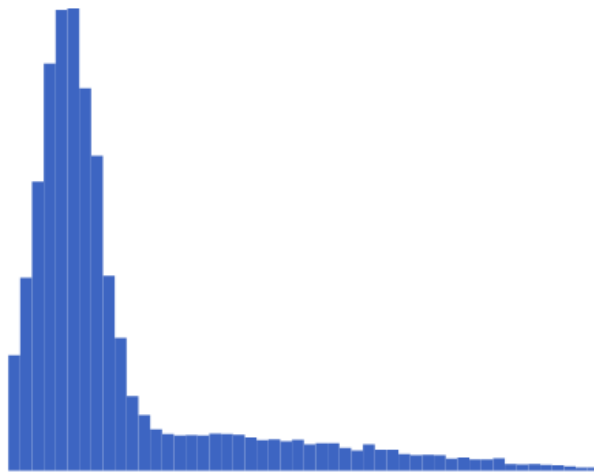
Большинство наборов данных, с которыми мы будем иметь дело, будут одномодальными (один пик). Мы можем дополнительно классифицировать унимодальные распределения, описывая, где большая часть чисел относится к пику.

Симметричный набор данных имеет равные объемы данных по обе стороны от пика. Обе стороны должны выглядеть примерно одинаково.



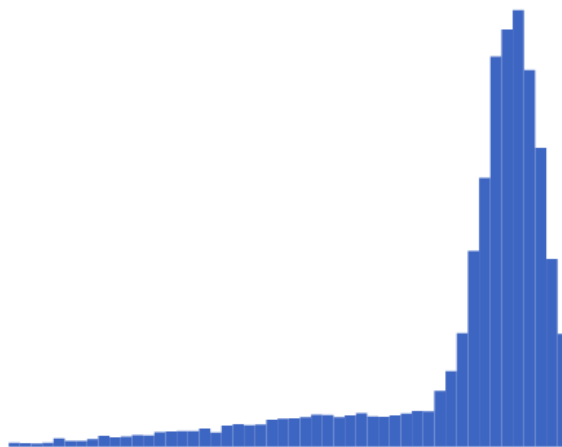
Набор данных с наклоном вправо имеет длинный хвост справа от пика, но большая часть данных находится слева.

Skew-Right



Набор данных с наклоном влево имеет длинный хвост слева от пика, но большая часть данных находится справа.

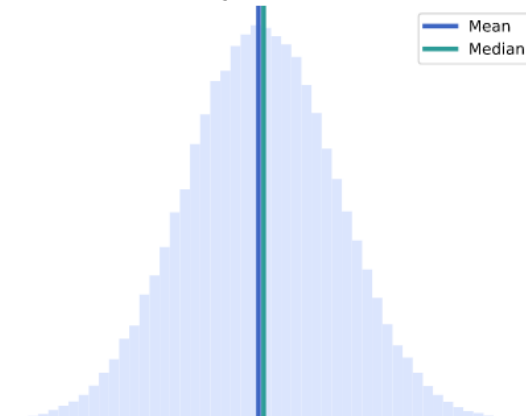
Skew-Left



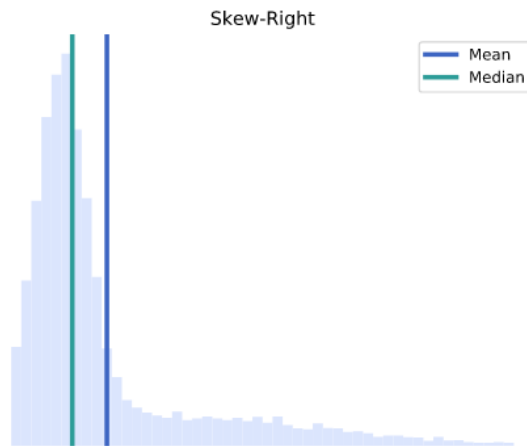
Тип распределения влияет на положение среднего и медианы. В сильно искаженных распределениях среднее значение становится менее полезным измерением.

СИММЕТРИЧНЫЙ

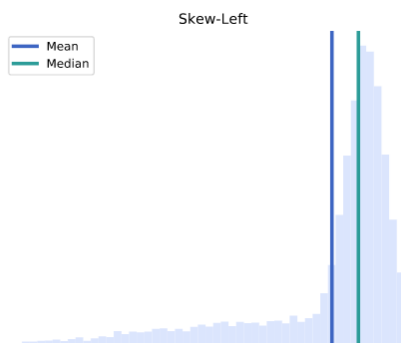
Symmetric



НАКЛОН ВПРАВО

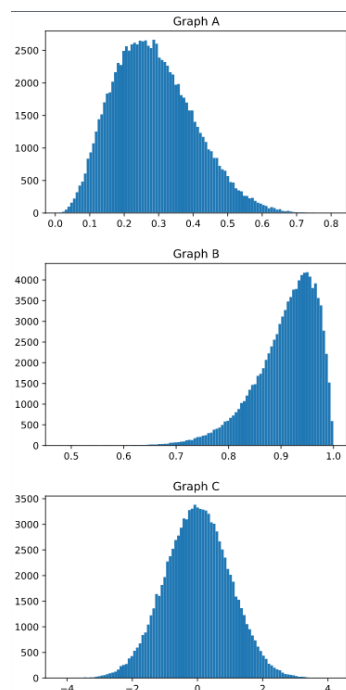


НАКЛОН ВЛЕВО



Задание

1. Определите форму распределения Графика А и сохраните свой ответ (в виде строки) в переменной `graph_a`.
2. Определите форму распределения Графика В и сохраните свой ответ (в виде строки) в переменной `graph_b`.
3. Определите форму распределения Графика С и сохраните свой ответ (в виде строки) в переменной `graph_c`.



Нормальное распределение, часть I

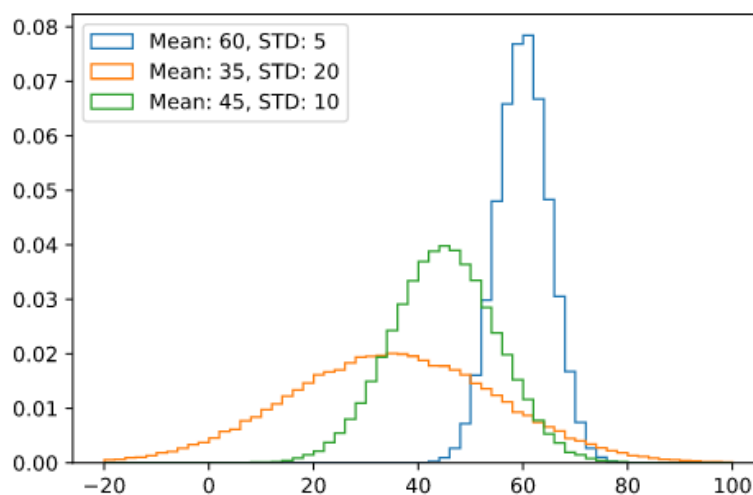
Наиболее распространенное распределение в статистике известно как нормальное распределение, которое является симметричным унимодальным распределением.

Многие вещи подчиняются нормальному распределению:

- Рост большой группы людей
- Измерение артериального давления для группы здоровых людей
- Ошибки в измерениях

Нормальные распределения определяются их средним значением и стандартным отклонением. Среднее значение устанавливает «сердину» распределения, а стандартное отклонение - «ширину» распределения. Большее стандартное отклонение приводит к более широкому распределению. Меньшее стандартное отклонение приводит к более тонкому распределению.

Вот несколько примеров нормального распределения с разными средними и стандартными отклонениями:



Как мы видим, каждый набор данных имеет одинаковую «форму», но с небольшими различиями в зависимости от их среднего значения и стандартного отклонения.

Нормальное распределение, часть 2

Мы можем создавать наши собственные нормально распределенные наборы данных с помощью NumPy. Использование этих наборов данных может помочь нам лучше понять свойства и поведение различных распределений. Мы также можем использовать их для моделирования результатов, которые затем можно использовать для сравнения с реальными данными.

Чтобы создать эти наборы данных, нам нужно использовать генератор случайных чисел. В библиотеке NumPy есть несколько функций для генерации случайных чисел, в том числе одна, специально созданная для генерации набора чисел, соответствующих нормальному

распределению: `np.random.normal`. Эта функция принимает следующие аргументы ключевого слова:

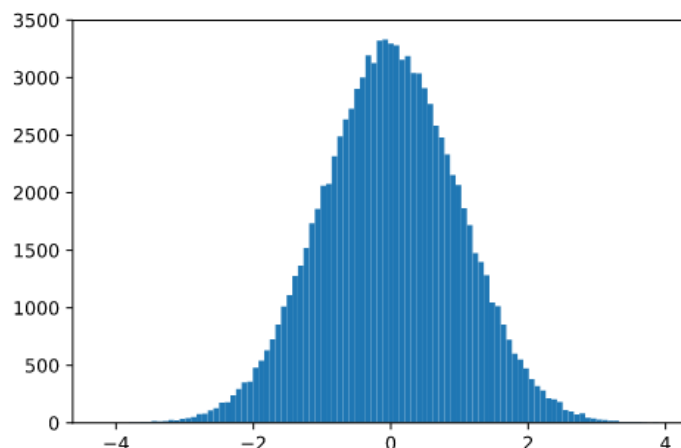
loc: среднее значение нормального распределения

scale: стандартное отклонение распределения

size: количество случайных чисел для генерации

```
a = np.random.normal(0, 1, size=100000)
```

Если бы мы изобразили этот набор случайных чисел в виде гистограммы, это выглядело бы так:



Задание

1. Наш друг - палеонтолог. Он изучает два типа динозавров: брахиозавр и фикозавр. Он говорит нам, что:

Брахиозавр имеет бедренную кость (бедренную кость) со средней длиной 6,7 футов и стандартным отклонением 0,7 фута.

У фикозавров есть бедренные кости (бедренная кость) со средней длиной 7,7 футов и стандартным отклонением 0,3 фута.

Мы хотели бы знать, как выглядят эти распределения.

Используйте `np.random.normal`, чтобы заполнить `b_data` (брахиозавр) и `f_data` (fictionosaurus) случайно сгенерированными выборками с правильным средним значением и стандартным отклонением. В каждом наборе данных должно быть 1000 выборок.

Код, который мы предоставили ниже, будет отображать распределения для вас.

```
import codecademylib
import numpy as np
from matplotlib import pyplot as plt

# Brachiosaurus
b_data =

# Fictionosaurus
f_data =
```

```
plt.hist(b_data,
         bins=30, range=(5, 8.5), histtype='step',
         label='Brachiosaurus')
plt.hist(f_data,
         bins=30, range=(5, 8.5), histtype='step',
         label='Fictionosaurus')
plt.xlabel('Femur Length (ft)')
plt.legend(loc=2)
plt.show()
```

2. Наш друг обнаружил бедренную кость неопознанного динозавра на его последних раскопках. Бедренная кость 6,6 футов в длину.

Ваш друг почти уверен, что бедренная кость принадлежит либо брахиозавру, либо фиктиозавру. Глядя на графические данные, определите, какому динозавру, скорее всего, принадлежит этот скелет, и сохраните свой ответ в переменной `mystery_dino`.

3. Наш друг обнаружил еще одну бедренную кость. Это бедро составляет 7,5 футов в длину. Очевидно, 7,5 футов ближе к среднему значению для фикозавра (7,7 футов), чем к среднему значению для брахиозавра (6,5 футов).

Можем ли мы быть абсолютно уверены в том, что бедренная кость принадлежит фиктиозавру? Сохраните свой ответ (True или False), чтобы ответить.

Как гистограммы помогают вам объяснить свой ответ?